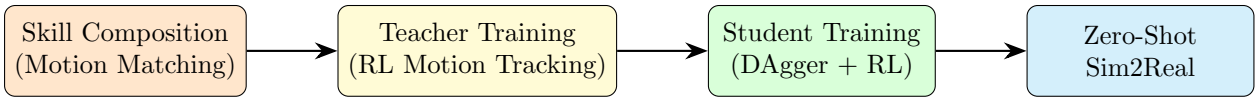
Lecture Notes: Motion Matching Pipeline in Perceptive Humanoid Parkour (PHP)

Reference: Wu, Huang, Yang et al. (2025)

Contents

1 Overview: Where Motion Matching Fits

The PHP framework has four stages:



Motion matching is an **offline kinematic trajectory synthesis** module. Its job:

- Take a sparse library of retargeted human motion clips (locomotion + parkour skills).
- Compose them into a *large, diverse* set of long-horizon reference trajectories of the form

$$\text{Locomotion} \rightarrow \text{Parkour Skill} \rightarrow \text{Locomotion}.$$

- These trajectories are then used as kinematic references for downstream RL training.

2 Motion Database Representation

2.1 Frame-Level Data

All motion clips are first retargeted to the 29-DOF Unitree G1 robot via OmniRetarget. Each frame i stores a robot configuration:

$$\mathbf{q}_i = (\mathbf{p}_i,\ r_i,\ \theta_i),$$

where

- $\mathbf{p}_i \in \mathbb{R}^3$: root (pelvis) translation,
- $r_i \in \mathbb{R}^4$: root quaternion orientation,
- $\theta_i \in \mathbb{R}^{29}$: joint angles.

2.2 Matching Feature Vector

For each frame i, a **matching feature** $\mathbf{x}_i \in \mathbb{R}^{27}$ is precomputed in the character's *local coordinate frame*. It concatenates three groups:

Feature vector $\mathbf{x}_i \in \mathbb{R}^{27}$

1. **Future root trajectory $\mathbf{t}_i \in \mathbb{R}^{12}$:**
   Planar root positions (x, y) and facing directions $(\cos\psi, \sin\psi)$ at horizons $\tau \in \{0.33,\ 0.67,\ 1.0\}$ s.
   $\Rightarrow 3 \times 4 = 12$ dimensions.
2. **Local foot state $\mathbf{f}_i \in \mathbb{R}^{12}$:**
   Left and right foot positions (3D) and linear velocities (3D), expressed in root frame.
   $\Rightarrow 2 \times (3 + 3) = 12$ dimensions.
3. **Root velocity $\mathbf{h}_i \in \mathbb{R}^3$:**
   Root linear velocity in the local frame.

All motion clips are also **mirrored** (left $\leftrightarrow$ right) to double the database size. For parkour clips, a box-shaped terrain asset is manually fitted and aligned with each motion.

2.3 Databases

Two kinds of databases are maintained:

- **Locomotion database** $\mathcal{D}_{\text{loco}} = \{(\mathbf{x}_i^{\text{loco}}, \mathbf{q}_i^{\text{loco}})\}$: standing, walking, running at 0.8–3.5 m/s.

- **Skill databases** $\{\mathcal{D}_k\}$, one per parkour skill k (step, climb, vault, etc.).

For each atomic skill clip in $\mathcal{D}_k$, two frame indices are *manually annotated*:

- s_k: skill start frame (where the main contact-rich maneuver begins),
- e_k: skill end frame.

A **pre-skill entry window** is defined with skill-dependent length H_k:

$$\boxed{\mathcal{E}_k := [s_k - H_k,\ s_k]} \tag{1}$$

This window covers the approach phase just before the maneuver (e.g. the final steps before takeoff in a vault).

3 Core Algorithm: Nearest-Neighbor Search

3.1 Query Feature Construction

At runtime (or synthesis time), given:

- current robot configuration $\mathbf{q}_t$,
- a 2D velocity command $\mathbf{u}_t^{\text{cmd}} \in \mathbb{R}^2$,

we construct a query feature $\hat{\mathbf{x}}_t$ by:

1. **Pose-based part**: extract foot state $\hat{\mathbf{f}}_t$ and root velocity $\hat{\mathbf{h}}_t$ from $\mathbf{q}_t$.
2. **Command-based part**: convert $\mathbf{u}_t^{\text{cmd}}$ into a future trajectory $\hat{\mathbf{t}}_t$ using a *critically damped spring model* (see Section 4).
3. Concatenate: $\hat{\mathbf{x}}_t = [\hat{\mathbf{t}}_t;\ \hat{\mathbf{f}}_t;\ \hat{\mathbf{h}}_t] \in \mathbb{R}^{27}$.

3.2 Nearest-Neighbor Retrieval

The best matching frame is found by:

$$\boxed{i_t^\star = \arg\min_{i \in \mathcal{C}_t} \|\hat{\mathbf{x}}_t - \mathbf{x}_i\|^2} \tag{2}$$

where $\mathcal{C}_t$ is the **search window** (candidate set), which varies by mode:

- **Locomotion mode**: $\mathcal{C}_t = \mathcal{D}_{\text{loco}}$ (entire locomotion database).
- **Skill transition mode**: $\mathcal{C}_t = \mathcal{E}_k$ (pre-skill entry window of skill k).

This search is triggered:

- Periodically, every M frames, **or**
- When the commanded velocity changes significantly.

After selecting $i_t^\star$, playback transitions to frame $i_t^\star$ and advances sequentially until the next search. A **short blending window** (inertialization) is applied at transitions to avoid discontinuities.

4 Critically Damped Spring Model

The spring model converts the velocity command into smooth future trajectory predictions.

Critically Damped Spring – Closed Form

Let s be the spring position, $\dot{s}$ its velocity, s_{goal} the target, and $y > 0$ the damping parameter. Define:

$$j_0 = s_0 - s_{\text{goal}}, \qquad j_1 = \dot{s}_0 + y\, j_0.$$

Then the state at future time τ:

$$\boxed{s(\tau) = e^{-y\tau}\big(j_0 + \tau\, j_1\big) + s_{\text{goal}}} \tag{3}$$

4.1 Planar Position from Target Velocity

Here the spring operates in *velocity space*: the "spring position" is the planar velocity, and we integrate to get positions.

With target velocity $\mathbf{u}_t^{\text{cmd}}$ and initial state $(\mathbf{p}_0, \dot{\mathbf{p}}_0)$, the future position is:

$$\boxed{p(\tau) = p_0 - \frac{j_1}{y^2}\, e^{-y\tau} + \frac{-j_0 - \tau\, j_1}{y}\, e^{-y\tau} + \frac{j_1}{y^2} + \frac{j_0}{y} + \mathbf{u}_t^{\text{cmd}}\, \tau} \tag{4}$$

applied component-wise in the plane (2D).

4.2 Heading Direction from Target Heading

For heading rotation, apply the spring *directly to the heading angle* ψ toward the target:

$$\psi_{\text{goal}} = \operatorname{atan2}\big(u_{t,y}^{\text{cmd}},\ u_{t,x}^{\text{cmd}}\big).$$

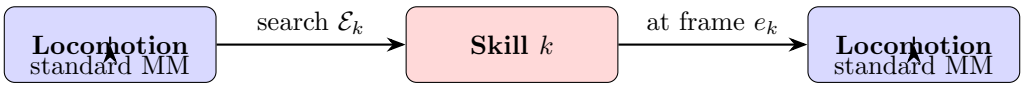
Evaluate $\psi(\tau)$ via Eq. (3) at $\tau \in \{0.33, 0.67, 1.0\}$ s. No integration is needed (the spring is in angle space directly).

4.3 Assembling the Future Trajectory Feature

Evaluate at $\tau \in \{0.33, 0.67, 1.0\}$ s, transform the resulting positions $p(\tau)$ and facing directions $(\cos\psi(\tau), \sin\psi(\tau))$ into the character's local frame $\Rightarrow$ form $\hat{\mathbf{t}}_t \in \mathbb{R}^{12}$.

5 Long-Horizon Trajectory Composition

The composition follows a three-phase state machine:



Algorithm 1: Long-Horizon Parkour Trajectory Synthesis via Motion Matching

Input : Locomotion database $\mathcal{D}_{\text{loco}}$; skill databases $\{\mathcal{D}_k\}$ with annotations (s_k, e_k, H_k); target skill index k; 2D velocity command schedule $\{\mathbf{u}_t^{\text{cmd}}\}$.

Output: Long-horizon kinematic trajectory $\{(\mathbf{q}_t, \mathbf{u}_t^{\text{cmd}})\}_{t=0}^{T}$.

`// Phase 1: Locomotion (approach)`

Initialize from standing pose;
mode $\leftarrow$ `LOCOMOTION`, frame_counter $\leftarrow 0$;
while *mode* $=$ ***LOCOMOTION and*** *skill transition not triggered* **do**
  if *frame_counter* $\bmod M = 0$ ***or*** *velocity command changed* **then**
    Construct query $\hat{\mathbf{x}}_t$ from current $\mathbf{q}_t$ and $\mathbf{u}_t^{\text{cmd}}$; `// Sec. 3.1`
    $i_t^\star \leftarrow \arg\min_{i \in \mathcal{D}_{\text{loco}}} \|\hat{\mathbf{x}}_t - \mathbf{x}_i\|^2$;
    Transition playback to frame $i_t^\star$ with inertialization blending;
  Advance playback: $\mathbf{q}_{t+1} \leftarrow \mathbf{q}_{i_t^\star+1}$;
  Record $(\mathbf{q}_t, \mathbf{u}_t^{\text{cmd}})$;
  frame_counter += 1;

`// Phase 2: Locomotion → Skill transition`

Restrict search to entry window: $\mathcal{C}_t \leftarrow \mathcal{E}_k = [s_k - H_k,\ s_k]$;
$i_t^\star \leftarrow \arg\min_{i \in \mathcal{E}_k} \|\hat{\mathbf{x}}_t - \mathbf{x}_i\|^2$;
Transition to frame $i_t^\star$ with inertialization blending;
Place terrain: apply terrain-to-root offset at frame $i_t^\star$ to current root pose;
Set command to straight (0°), keep same speed level;

`// Phase 3: Skill execution (sequential playback, no MM)`

for *frame* $= i_t^\star$ **to** e_k **do**
  Advance playback sequentially (no motion matching);
  Record $(\mathbf{q}_t, \mathbf{u}_t^{\text{cmd}})$;

`// Phase 4: Skill → Locomotion transition`

$\mathcal{C}_t \leftarrow \mathcal{D}_{\text{loco}}$;
$i_t^\star \leftarrow \arg\min_{i \in \mathcal{D}_{\text{loco}}} \|\hat{\mathbf{x}}_t - \mathbf{x}_i\|^2$;
Continue locomotion for additional $\sim 2\,\text{s}$, then stop;

6 Transition Smoothing: Inertialization

When switching playback to a newly retrieved frame, a sudden jump in pose would occur. **Inertialization** smooths this:

1. At the transition instant, compute the **offset** Δ between the currently playing motion and the target motion.
2. After switching, apply Δ as an additive correction so the output is continuous.
3. Decay $\Delta \to 0$ using the critically damped spring (Eq. (3)) with $s_{\text{goal}} = 0$.

In code, for each DOF or root component:

$$\Delta(\tau) = e^{-y\tau}\big(\Delta_0 + \tau\,(\dot{\Delta}_0 + y\,\Delta_0)\big),$$

where Δ_0 is the initial offset and $\dot{\Delta}_0$ is the initial offset velocity.

7 Dataset Construction Details

7.1 Velocity Command Sampling

Each synthesized trajectory uses:

- **Speed levels**: low (1 m/s), high (2 m/s).
- **Turning directions**: $\{-90°, -45°, 0°, 45°, 90°\}$.

During skill execution, the command is set to straight (0°) at the same speed. After skill ends, locomotion continues for 2 s, then stops.

7.2 Transition Density via Approach Distance Randomization

To prevent non-causal shortcuts, the **pre-skill locomotion duration** is randomized:

$$t_{\text{pre-skill}} \sim \text{Uniform}(0.1,\ 3.0)\ \text{s}, \quad \text{average interval } 0.3\ \text{s}.$$

Different approach distances force the motion matching engine to select different stride sequences, yielding distinct entry poses (e.g. left-leg vs. right-leg lead). This diversity is critical for learning a policy that generalizes across varying distances.

7.3 Terrain Randomization

Around each synthesized trajectory, obstacle geometry is perturbed:

- Width: sampled from minimum required up to 1.5 m.
- Other dimensions: ±5 cm perturbation.
- Yaw: ±45° randomization.

7.4 Distractors

Random distractor boxes are placed near the reference trajectory to improve robustness and reduce overfitting to the specific obstacle layout.

8 Pseudocode Summary for Implementation

Implementation Checklist

1. **Retarget motions** to robot DOFs → frame sequences $\{(\mathbf{p}_i, r_i, \theta_i)\}$.
2. **Precompute features** $\mathbf{x}_i \in \mathbb{R}^{27}$ for every frame (future traj + feet + root vel).
3. **Mirror** all clips (swap left/right) to double database.
4. **Annotate** each skill clip: (s_k, e_k, H_k).
5. **Implement spring model** (Eqs. (3), (4)) for query trajectory generation.
6. **Implement NN search** (Eq. (2)) with brute-force or KD-tree.
7. **Implement inertialization** for smooth blending at transitions.
8. **Run the composition loop** (Algorithm 1) with sampled velocity commands, approach durations, and terrain randomizations.
9. **Record** paired trajectories $\{(\mathbf{q}_t, \mathbf{u}_t^{\text{cmd}})\}$ for policy training.

9 Key Equations at a Glance

Concept	Equation	Ref.
Feature vector	$\mathbf{x}_i = [\mathbf{t}_i;\ \mathbf{f}_i;\ \mathbf{h}_i] \in \mathbb{R}^{27}$	Sec. 2.2
Entry window	$\mathcal{E}_k = [s_k - H_k,\ s_k]$	Eq. (1)
NN retrieval	$i_t^\star = \arg\min_{i \in \mathcal{C}_t} \|\hat{\mathbf{x}}_t - \mathbf{x}_i\|^2$	Eq. (2)
Spring (angle/vel)	$s(\tau) = e^{-y\tau}(j_0 + \tau j_1) + s_{\text{goal}}$	Eq. (3)
Spring (position)	$p(\tau) = p_0 - \frac{j_1}{y^2}e^{-y\tau} + \frac{-j_0 - \tau j_1}{y}e^{-y\tau} + \frac{j_1}{y^2} + \frac{j_0}{y} + \mathbf{u}^{\text{cmd}}\tau$	Eq. (4)
Inertialization	$\Delta(\tau) = e^{-y\tau}(\Delta_0 + \tau(\dot{\Delta}_0 + y\Delta_0))$	Sec. 6

10 Skill List (from the Paper)

Category	Skill	Clip Duration (s)
Locomotion	Stand / Walk / Run (0.8–3.5 m/s)	495.5
4*@ 1.0 m/s	Step (36 cm)	2.2
	Climb (58 cm)	12.1
	Climb (76 cm)	8.8
	Climb (94 cm)	10.3
6*@ 2.0 m/s	Step (36 cm)	1.6
	Climb (58 cm)	6.1
	Climb (76 cm)	4.4
	Climb (94 cm)	5.2
	Climb (125 cm)	5.9
	Dash Vault / Speed Vault	5.0 / 3.1
@ 3.0 m/s	Cat Vault	1.5

Note: parkour skill clips are very short (1.5–12 s each). Motion matching is what densifies these into a large diverse training set.